

# CONECTANDO JAVA COM BANCO DE DADOS

## 1. Por que usar um banco de dados?

Muitos sistemas precisam manter as informações com as quais eles trabalham, seja para permitir consultas futuras, geração de relatórios ou possíveis alterações nas informações. Para que esses dados sejam mantidos para sempre, esses sistemas geralmente guardam essas informações em um banco de dados, que as mantém de forma organizada e prontas para consultas.

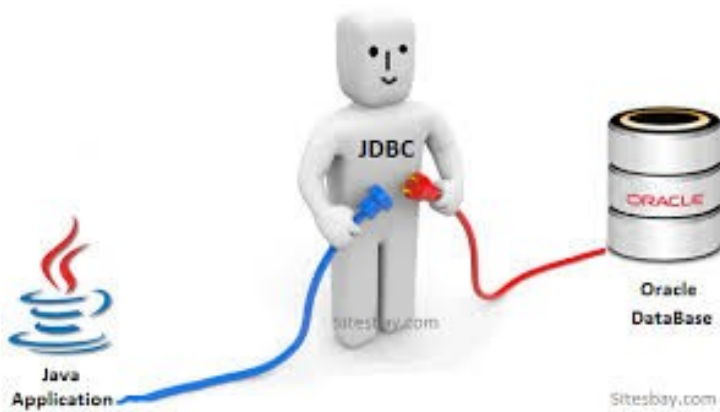
A maioria dos bancos de dados comerciais são os chamados relacionais, que é uma forma de trabalhar e pensar diferente ao paradigma orientado a objetos.

O processo de armazenamento de dados é também chamado de **persistência**.

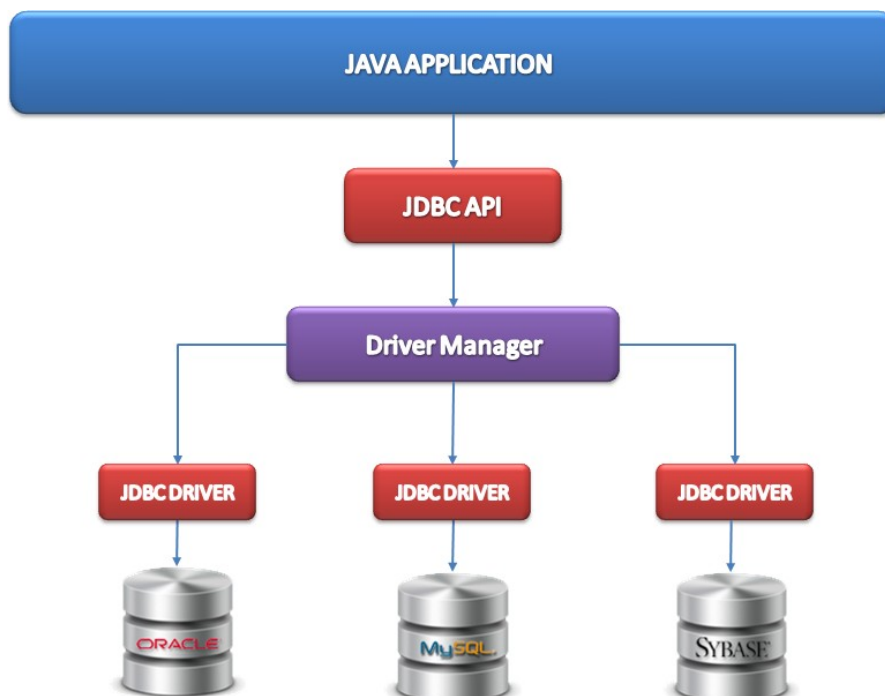
A linguagem Java possui classes que permitem a conexão com um banco de dados.

## 2. JDBC

Essa biblioteca de classes de persistência em banco de dados relacionais do Java fazem parte do pacto JDBC (Java Database Connectivity).



O JDBC é uma API (Aplication Program Interface) que permite a comunicação com diversos banco de dados SQL (Oracle, MySQL, produtos Microsoft, etc).



### O que é então o JDBC?

- consiste em uma biblioteca
- implementada em Java
- disponibiliza classes e interfaces para o acesso ao banco de dados

**Para cada banco de dados existe uma implementação:  
Um driver, no caso uma interface que deve ser implementada**

São interfaces porque levam em considerações particularidades

### 3. Pacote java.sql

- fornece a API para acesso e processamento de dados;
- geralmente acessa uma base de dados relacional;
- principais classes e interfaces:
  - **DriverManager**, responsável por criar uma conexão com o banco de dados;
  - **Connection**, classe responsável por manter uma conexão aberta com o banco;
  - **Statement**, gerencia e executa instruções SQL;
  - **PreparedStatement**, gerencia e executa instruções SQL, permitindo também a passagem de parâmetros em uma instrução;
  - **ResultSet**, responsável por receber os dados obtidos em uma pesquisa ao banco.

#### 3.1 A conexão com o banco de dados

##### DriverManager

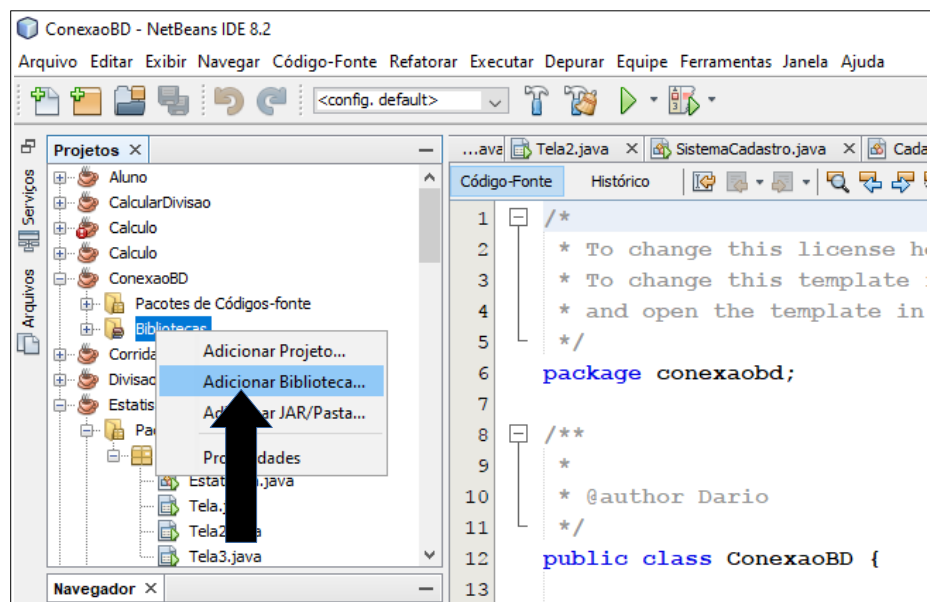
- responsável pelo gerenciamento de Drivers JDBC
- estabelece conexões a banco de dados

##### Connection

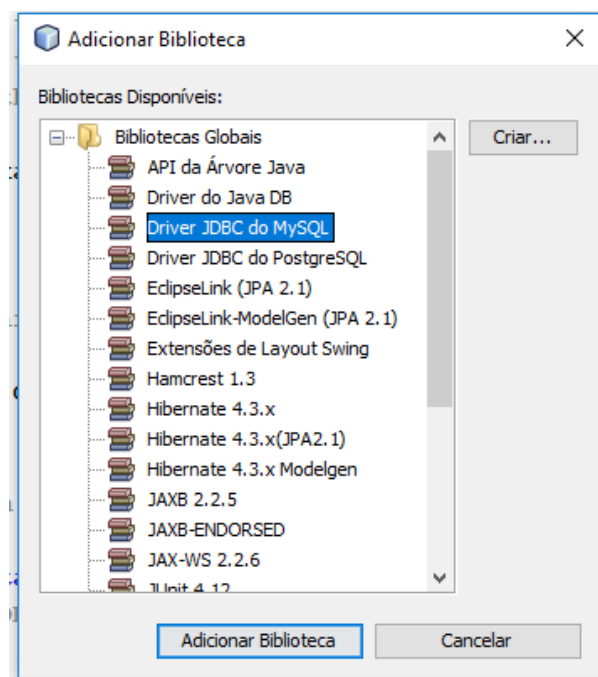
- representa a conexão com o banco de dados
- proporcionar informações sobre as tabelas do banco através de transações

Como vamos conectar com o banco de dados MySQL precisamos carregar no NetBeans o Driver específico do banco de dados MySQL.

Para isso, no NetBeans, após criado o projeto clicamos o botão direito da pasta Bibliotecas do projeto em que estamos e aparecerá um menu e no mesmo escolhemos a opção Adicionar Bibliotecas (conforme seta na Figura a seguir).



Será exibida então a janela Adicionar Biblioteca, selecionamos nela a opção Driver JDBC do MySQL, conforme figura a seguir.



**Após, clicamos no botão Adicionar Biblioteca e pronto, o Driver já está acessível ao projeto.**

**Assim, para testarmos a conexão com o banco de dados podemos fazer o seguinte código:**

```
package conexaobd;
import java.sql.*;
import javax.swing.*;
import java.sql.DriverManager;
import java.util.logging.Level;
import java.util.logging.Logger;

public class ConexaoBD {

    public static void main(String[] args) {
        final String driver = "com.mysql.jdbc.Driver";
        final String url = "jdbc:mysql://localhost:3306/pessoa";
        try {
            Class.forName(driver);
            Connection connection = DriverManager.getConnection(url,"root","");
            JOptionPane.showMessageDialog(null,"Conexão realizada com sucesso");
            connection.close();

        } catch (ClassNotFoundException ex) {
            JOptionPane.showMessageDialog(null,"Driver JDBC não encontrado!");
        }
        catch (SQLException ex){
            JOptionPane.showMessageDialog(null,"Problemas na conexão com a fonte de dados");
        }
    }
}
```

-----

### 3.2 Criação de um banco de dados

#### Statement

- Implementação de uma Interface que fornece métodos para executar uma instrução SQL;
- Não aceita a passagem de parâmetros;
- principais métodos da Interface Statement são (SUN, 2007):
  - executeUpdate(), executa instruções SQL do tipo: INSERT, UPDATE e DELETE;
  - execute(), executa instruções SQL de busca de dados do tipo SELECT;
  - close(), libera o recurso que estava sendo utilizado pelo objeto.

**Assim, para testarmos a criação de um banco de dados e a classe Statement podemos fazer o seguinte código:**

```
public void geraDb(){
    try {
        Class.forName("com.mysql.jdbc.Driver");
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/mysql",
"root","erechimlkh");

        Statement st = con.createStatement();
        String sql = "CREATE DATABASE IF NOT EXISTS funcionario";
        st.executeUpdate(sql);
        JOptionPane.showMessageDialog(null,"Banco de dados criado com sucesso");
        con.close();

    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null,"Problemas na conexão com a fonte de dados");
    } catch (ClassNotFoundException ex) {
        JOptionPane.showMessageDialog(null,"Driver JDBC não encontrado!");
    }
}
```